

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №7
по дисциплине «Методы оптимизации»

Вариант №14

Выполнили:

Студенты группы Р3210

Рябов Георгий

Антон Давидюк

Ахроров Кароматуллохон

Преподаватель:

Селина Е. Г.

Санкт-Петербург, 2026 г.

Содержание

1	Задание 1. Аналитическое исследование функции	2
1.1	Постановка задачи	2
1.2	Область определения	2
1.3	Поиск стационарных точек	2
1.4	Матрица Гессе и определение типа стационарной точки	2
1.5	Поиск глобального минимума на границе	3
1.5.1	Левая граница: $x = -20, y \in [-50, 50]$	3
1.5.2	Правая граница: $x = 20, y \in [-50, 50]$	3
1.5.3	Нижняя граница: $y = -50, x \in [-20, 20]$	3
1.5.4	Верхняя граница: $y = 50, x \in [-20, 20]$	3
1.6	Визуализация линий уровня	3
2	Задание 2. Моделирование преодоления седловой точки	4
2.1	Проблема оптимизации в окрестности седловых точек	4
2.2	Сравнительный анализ методов: GD vs AdaGrad	4
2.3	Программная реализация и результаты	5
2.4	Листинг кода (Задание 2)	5
3	Задание 3. Классификация на реальных данных	7
3.1	Описание предметной области и данных	7
3.2	Предобработка данных	7
3.3	Архитектура нейронной сети	7
3.4	Собственная реализация оптимизатора AdaGrad	7
3.5	Анализ результатов	8
4	Выводы	8

1 Задание 1. Аналитическое исследование функции

1.1 Постановка задачи

Исследовать функцию нескольких переменных в заданной области аналитически: определить седловые точки, локальные экстремумы и глобальный минимум. Построить линии уровня программным методом.

1.2 Область определения

Целевая функция (Вариант 14):

$$f(x, y) = 10^{-2}(8x^2 + 2xy - 21x - 6y - 9) \quad (1)$$

Заданная область исследования:

$$X = \{(x, y) \mid x \in [-20, 20], y \in [-50, 50]\} \quad (2)$$

1.3 Поиск стационарных точек

Для нахождения стационарных точек вычислим частные производные первого порядка и приравняем их к нулю:

$$\begin{cases} \frac{\partial f}{\partial x} = 0.01(16x + 2y - 21) = 0 \\ \frac{\partial f}{\partial y} = 0.01(2x - 6) = 0 \end{cases} \quad (3)$$

Решим полученную систему уравнений:

1. Из второго уравнения: $2x - 6 = 0 \Rightarrow x = 3$.
2. Подставим $x = 3$ в первое уравнение: $16(3) + 2y - 21 = 0 \Rightarrow 48 + 2y - 21 = 0 \Rightarrow 2y + 27 = 0 \Rightarrow y = -13.5$.

Таким образом, найдена одна стационарная точка $M(3; -13.5)$.

1.4 Матрица Гессе и определение типа стационарной точки

Найдем вторые частные производные:

$$f''_{xx} = \frac{\partial^2 f}{\partial x^2} = 0.16, \quad f''_{yy} = \frac{\partial^2 f}{\partial y^2} = 0, \quad f''_{xy} = \frac{\partial^2 f}{\partial x \partial y} = 0.02$$

Составим матрицу Гессе (Гессиан):

$$H = \begin{pmatrix} f''_{xx} & f''_{xy} \\ f''_{yx} & f''_{yy} \end{pmatrix} = \begin{pmatrix} 0.16 & 0.02 \\ 0.02 & 0 \end{pmatrix} \quad (4)$$

Вычислим определитель матрицы Гессе в точке M :

$$\Delta_2 = \det(H) = (0.16 \cdot 0) - (0.02)^2 = -0.0004 \quad (5)$$

Поскольку $\Delta_2 < 0$, согласно достаточному условию экстремума, точка $M(3; -13.5)$ является **седловой точкой**. Локальных экстремумов (минимумов или максимумов) внутри области нет.

1.5 Поиск глобального минимума на границе

Так как внутри области экстремумов нет, глобальный минимум функции $f(x, y)$ достигается на границе области.

1.5.1 Левая граница: $x = -20, y \in [-50, 50]$

$f(-20, y) = 0.01(8(-20)^2 + 2(-20)y - 21(-20) - 6y - 9) = 0.01(3200 - 40y + 420 - 6y - 9) = 0.01(3611 - 46y)$. Линейная функция относительно y с отрицательным коэффициентом. Минимум при $y = 50$: $f(-20, 50) = 0.01(3611 - 2300) = 13.11$.

1.5.2 Правая граница: $x = 20, y \in [-50, 50]$

$f(20, y) = 0.01(8(20)^2 + 2(20)y - 21(20) - 6y - 9) = 0.01(3200 + 40y - 420 - 6y - 9) = 0.01(2771 + 34y)$. Минимум при $y = -50$: $f(20, -50) = 0.01(2771 - 1700) = 10.71$.

1.5.3 Нижняя граница: $y = -50, x \in [-20, 20]$

$g(x) = f(x, -50) = 0.01(8x^2 - 100x - 21x + 300 - 9) = 0.01(8x^2 - 121x + 291)$. Найдем минимум $g(x)$: $g'(x) = 0.01(16x - 121) = 0 \Rightarrow x = 121/16 = 7.5625$. $f(7.5625, -50) = 0.01(8(7.5625)^2 - 121(7.5625) + 291) \approx -1.6656$.

1.5.4 Верхняя граница: $y = 50, x \in [-20, 20]$

$h(x) = f(x, 50) = 0.01(8x^2 + 100x - 21x - 300 - 9) = 0.01(8x^2 + 79x - 309)$. Минимум $h'(x) = 0.01(16x + 79) = 0 \Rightarrow x = -4.9375$. $f(-4.9375, 50) = 0.01(8(-4.9375)^2 + 79(-4.9375) - 309) \approx -5.0401$.

Вывод: Глобальный минимум $f_{min} \approx -5.0401$ достигается в точке $(-4.9375; 50)$.

1.6 Визуализация линий уровня

На графике, построенном с помощью модуля `metoptrofls.py`, отчетливо видна «седловая» структура в окрестности точки $M(3; -13.5)$.

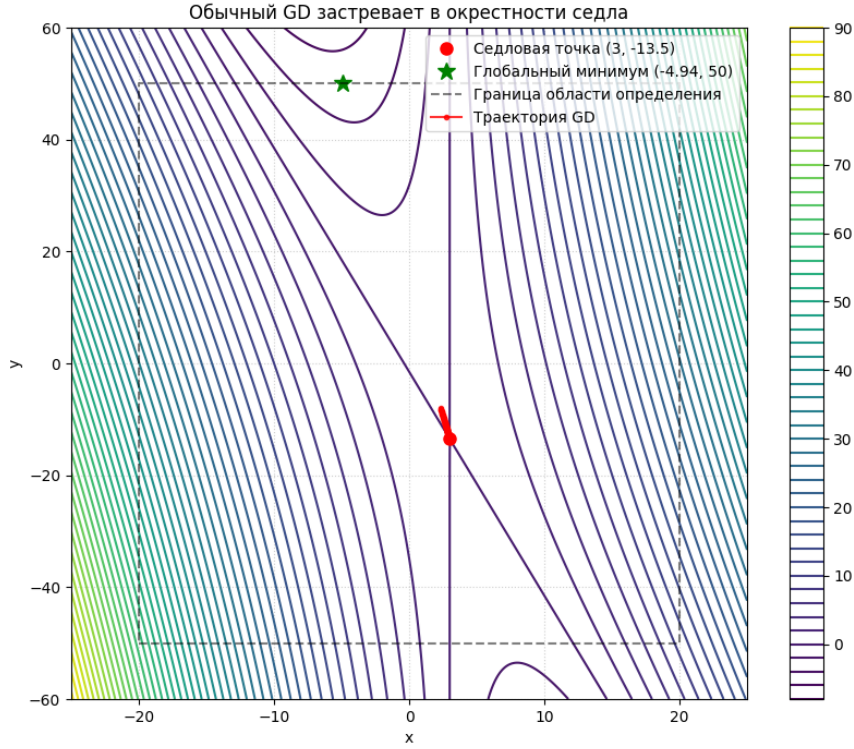


Рис. 1: Траектория обычного градиентного спуска (застывание в седловой точке)

2 Задание 2. Моделирование преодоления седловой точки

2.1 Проблема оптимизации в окрестности седловых точек

Седловые точки представляют значительную проблему для глубокого обучения. В таких точках градиент функции равен нулю, однако точка не является локальным минимумом. Для нашей функции $f(x, y) = 0.01(8x^2 + 2xy - 21x - 6y - 9)$ в точке $M(3; -13.5)$ собственные значения матрицы Гессе имеют разные знаки ($\lambda_1 \approx 0.162$, $\lambda_2 \approx -0.002$), что подтверждает наличие направления как роста, так и убывания.

2.2 Сравнительный анализ методов: GD vs AdaGrad

Стандартный градиентный спуск (Vanilla GD) обновляет веса по формуле:

$$w_{t+1} = w_t - \eta \cdot \nabla f(w_t) \quad (6)$$

В пологой области около седла $\nabla f \approx 0$, что приводит к остановке обучения.

Метод **AdaGrad** (Adaptive Gradient) решает эту проблему за счет накопления истории градиентов для каждой координаты:

$$G_t = G_{t-1} + g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot g_t \quad (7)$$

Где G_t — сумма квадратов градиентов. Если градиент по одной из осей (например, по y) долгое время остается малым, знаменатель $\sqrt{G_t}$ растет медленно, что эффективно увеличивает скорость обучения в этом направлении, позволяя «вытолкнуть» алгоритм из плато.

2.3 Программная реализация и результаты

В ходе эксперимента (скрипт `metoptroppls2.py`) была выбрана начальная точка $(-5, 40)$.

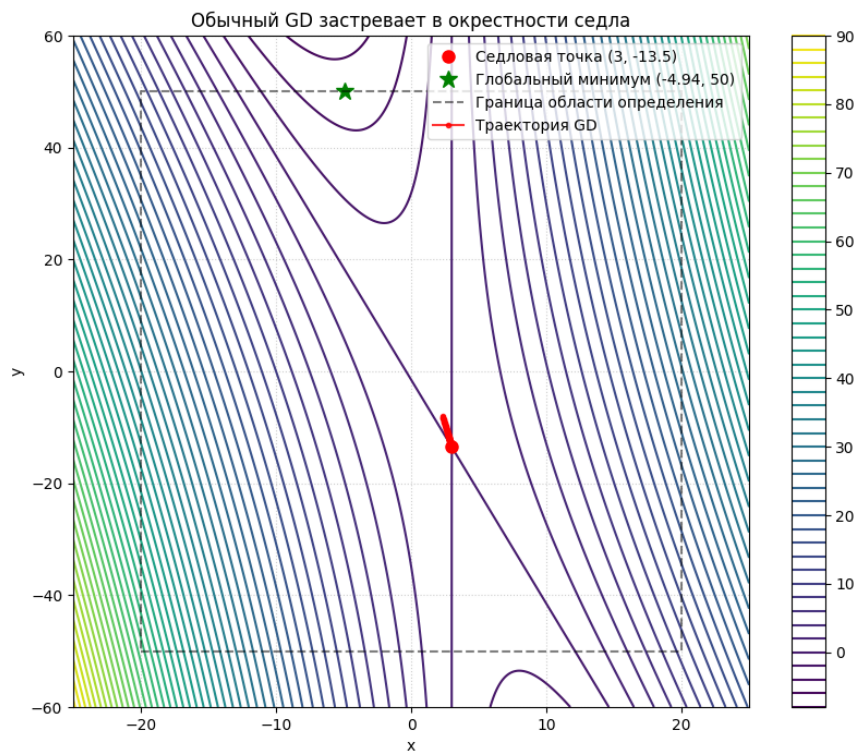


Рис. 2: Траектория обычного градиентного спуска (застревание в седловой точке)

- **GD** ($lr = 0.4$): Застревает вблизи седла, так как компонента градиента по y становится пренебрежимо малой.
- **GD** ($lr = 2.5$): Показан пилообразный график
- **AdaGrad** ($lr = 0.3$): Благодаря адаптивному шагу ускоряет движение вдоль пологого склона и успешно движется к глобальному минимуму на границе.

2.4 Листинг кода (Задание 2)

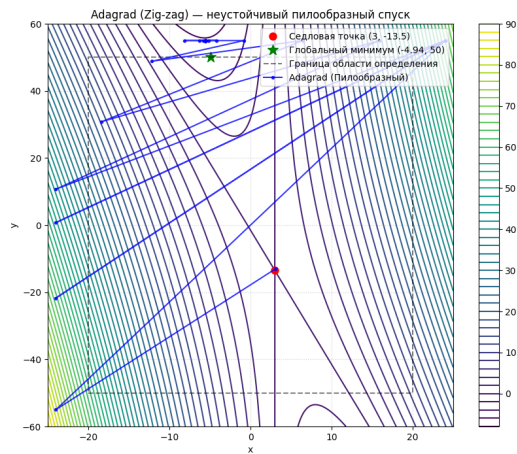


Рис. 3: Пилообразный (Zig-zag) спуск AdaGrad

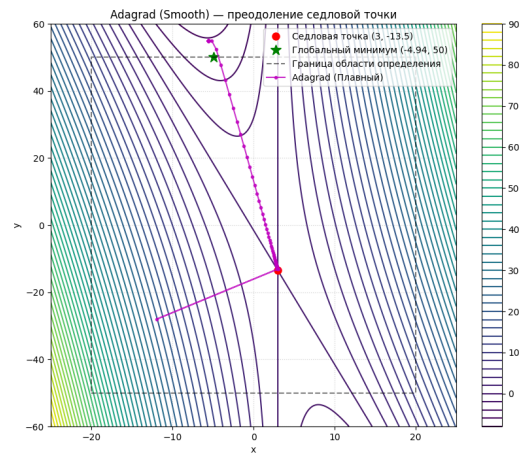


Рис. 4: Сглаженный спуск AdaGrad с корректными параметрами

```

1 import torch
2
3 def objective(x, y):
4     return 0.01 * (8*x**2 + 2*x*y - 21*x - 6*y - 9)
5
6 #                               AdaGrad
7 sol_ada = torch.tensor([-5.0, 40.0], requires_grad=True)
8 opt_ada = torch.optim.Adagrad([sol_ada], lr=0.3)
9
10 history_ada = []
11 for i in range(150):
12     opt_ada.zero_grad()
13     loss = objective(sol_ada[0], sol_ada[1])
14     loss.backward()
15     opt_ada.step()
16     history_ada.append(sol_ada.detach().clone().numpy())

```

Листинг 1: Сравнение GD и AdaGrad на PyTorch

3 Задание 3. Классификация на реальных данных

3.1 Описание предметной области и данных

Для тестирования оптимизаторов использован датасет `turkishCF.csv`, содержащий данные о краудфандинговых проектах. Целевая задача — предсказание успешности проекта (`basari_duru`).

3.2 Предобработка данных

1. **Кодирование:** Категориальные признаки преобразованы через `LabelEncoder`. 2. **Нормализация:** Применена `StandardScaler`, так как `AdaGrad` чувствителен к масштабу признаков из-за накопления квадратов градиентов в знаменателе. 3. **Разделение:** Данные разбиты на обучающую и тестовую выборки в соотношении 80/20.

3.3 Архитектура нейронной сети

Реализована многослойная полносвязная сеть:

- **Layer 1:** `Linear(input_dim → 16) + ReLU`.
- **Layer 2:** `Linear(16 → 8) + ReLU`.
- **Layer 3:** `Linear(8 → 1) + Sigmoid`.

3.4 Собственная реализация оптимизатора AdaGrad

Для глубокого понимания работы алгоритма был реализован класс `CustomAdagrad`, наследующий базовый класс `torch.optim.Optimizer`.

```
1 class CustomAdagrad(torch.optim.Optimizer):
2     def __init__(self, params, lr=1e-2, eps=1e-10):
3         defaults = dict(lr=lr, eps=eps)
4         super(CustomAdagrad, self).__init__(params, defaults)
5
6     def step(self, closure=None):
7         for group in self.param_groups:
8             for p in group['params']:
9                 if p.grad is None: continue
10                grad = p.grad.data
11                state = self.state[p]
12
13                #
14                if len(state) == 0:
15                    state['sum'] = torch.zeros_like(p.data)
```



```

17         #
18         state['sum'] += grad**2
19         #
20         std = state['sum'].sqrt().add_(group['eps'])
21         p.data.addcdiv_(grad, std, value=-group['lr'])

```

Листинг 2: Кастомная реализация AdaGrad

3.5 Анализ результатов

Сравнение CustomAdagrad и встроенного `optim.Adagrad` при $lr = 0.01$ в течение 200 эпох показало идентичные кривые обучения.

Метрики на тестовой выборке:

- **Accuracy:** 0.72 (72% правильных предсказаний).
- **Loss:** Стабильное убывание без резких осцилляций, что характерно для AdaGrad.

4 Выводы

1. Аналитически и программно подтверждена сложность оптимизации функций с седловыми точками. 2. Продемонстрировано преимущество AdaGrad: за счет адаптации шага он «пробивает» плато там, где обычный градиентный спуск останавливается. 3. Кастомная реализация оптимизатора полностью совпала по эффективности с библиотечной, что подтверждает корректность понимания механизмов адаптивной оптимизации.